

Experiment 6

Academic Year 2026-2027			
Program: Electronics Engineering (VLSI Design and Technology)			
Year of Study	Second Year	Semester	III
Course	Skill Lab: Python Programming	Course Code	VDCOR2VS201
Class	SY VLSI	Course In-Charge	Mr. Nirmol Munvar
Exp Name	Functions, default arguments, lambda functions		
CO	Analyze and implement control structures and user-defined functions to solve logical and numerical problems using Python.		

Functions

A **function** is a reusable block of code designed to perform a specific task. Functions help make programs modular, reduce code repetition, and improve readability.

A function is defined using the def keyword.

Basic syntax:

```
def function_name(parameters):
    statements
```

The function is executed by calling its name.

```
def greet():
    print("Hello")
```

```
greet()
```

Function with Parameters

Parameters allow data to be passed into a function.

```
def add(a, b):
    print(a + b)
```

```
add(10, 20)
```

Here, a and b are parameters, while 10 and 20 are arguments passed during the function call.

Function with Return Value

A function can return a result using the return statement.

```
def add(a, b):
    return a + b
```

```
result = add(10, 20)
```

```
print(result)
```

return sends the result back to the point where the function was called.

A function can have multiple parameters and can also return multiple values.

```
def calculate(a, b):
    return a + b, a - b
```

```
sum_value, difference = calculate(20, 10)
```

```
print(sum_value)
```

```
print(difference)
```

Default Arguments

A **default argument** is a parameter that has a predefined value. If the caller does not provide a value for that parameter, Python uses the default value.

Syntax:

```
def function_name(parameter=default_value):
```

```
    statements
```

Example:

```
def greet(name="Student"):
```

```
    print("Hello", name)
```

```
greet("Rahul")
```

```
greet()
```

Output:

Hello Rahul

Hello Student

Default arguments are useful when a parameter usually has a standard value but can be changed when required.

A function can have multiple default arguments.

```
def student(name, branch="VLSI", year=3):
```

```
    print(name, branch, year)
```

```
student("Rahul")
```

```
student("Priya", "ECE", 4)
```

Parameters with default values should generally be placed after parameters without default values.

```
def student(name, branch="VLSI"):
```

```
    print(name, branch)
```

Lambda Functions

A **lambda function** is a small anonymous function. It is useful when a simple function is required for a short operation.

Syntax:

lambda arguments: expression

Unlike a normal function defined using def, a lambda function normally contains a single expression.

Example:

```
square = lambda x: x * x
```

```
print(square(5))
```

Output:

25

Another example:

```
add = lambda a, b: a + b
```

```
print(add(10, 20))
```

Lambda functions are commonly used with functions such as map(), filter(), and sorted().

For example:

```
numbers = [1, 2, 3, 4, 5]
```

```
squares = list(map(lambda x: x * x, numbers))
```

```
print(squares)
```

A lambda function can also contain a conditional expression.

```
check = lambda x: "Even" if x % 2 == 0 else "Odd"
```

```
print(check(7))
```

For larger or more complex operations, a regular def function is generally more readable than a lambda function.

Basic Exercises

Exercise 1: Basic Functions

Write a Python program containing separate functions to:

- Calculate the sum of two numbers.
- Calculate the difference between two numbers.
- Calculate the product of two numbers.
- Calculate the division of two numbers.

Accept the values from the user and call the appropriate functions to display the results.

Exercise 2: Default Arguments

Write a function `student_details()` that accepts:

- Student name
- Branch, with "VLSI" as the default value
- Year, with 3 as the default value

Call the function using different combinations of arguments and display the student details.

Exercise 3: Lambda Functions

Given a list of numbers, use lambda functions to:

- Find the square of each number.
- Check whether each number is even or odd.
- Find the cube of each number.

Use `map()` where appropriate and display the resulting lists.

Exercise 4: Combined Function Exercise

Write a simple calculator using functions. Create separate functions for addition, subtraction, multiplication, and division.

Use **default arguments** for the division function to provide a default divisor of 1.

Use a **lambda function** to calculate the square of the final result.

For example, if the user enters 10 and 5 for addition, the program should calculate 15 and then use the lambda function to calculate its square, 225.









